WnW: Waxing-and-Waning KV Cache for Long-Form Speech LLMs

Yiming Yao[1*] **Chenyang Lyu**[2*] **Xuanfan Ni**[2] **Longyue Wang**[2]
Weihua Luo[2] **Yazheng Yang**[1,3] **Jinsong Su**[1,3†]
[1]School of Informatics, Xiamen University [2]Alibaba Group
[3]Key Laboratory of Digital Protection and Intelligent Processing of Intangible Cultural Heritage of Fujian and Taiwan (Xiamen University), Ministry of Culture and Tourism, China

Abstract

Long-form audio makes the Key-Value (KV) cache the dominant bottleneck in speech LLMs, yet existing KV compression methods commit to an irreversible retention set at prefill. However, we reveal a severe *attention mismatch* in this static paradigm particularly for long-form audio: prefill attention concentrates at the audio's start, whereas decode-time attention distributes broadly. This weak overlap between their top-attended positions leads to performance degradation, with generation collapsing into repetition at extreme compression rates. To bridge this gap, we propose **WnW (Waxing-and-Waning KV cache)**, which makes eviction reversible by partitioning KV-heads into three roles via offline calibration. *Anchor* heads keep all audio KV on GPU and provide a per-step signal of which audio region the current token needs; *tidal* heads, guided by that signal, recall evicted KV from CPU on demand; *fixed* heads discard theirs permanently. Evaluations on LibriSpeech-Long Benchmark across two 3B backbones (Voxtral-Mini and Qwen2.5-Omni) demonstrate that WnW preserves near-full-cache accuracy while maintaining only **20%** of audio tokens on GPU—a regime where irreversible methods fail to terminate. WnW exhibits strong generalization across languages, tasks, and domains, with CPU-GPU chunk swapping introducing negligible decoding overhead. More broadly, WnW transforms KV compression into a reversible memory hierarchy in which a small set of calibrated observer heads governs the placement of the remaining cache.

1 Introduction

Speech Large Language Models (speech LLMs) such as Voxtral (Liu et al., 2025a) can take audio inputs of arbitrary length and produce free-form text directly from raw waveforms, enabling the transcription of meetings, lectures, and long dialogues and, increasingly, speech translation (Wu et al., 2025; Li et al., 2026; Gao et al., 2026). Long-form audio shifts the inference bottleneck from the model's parameters to the key-value (KV) cache: at 12.5–25 audio tokens per second, hours-long audio spans hundreds of thousands of KV positions, and audio typically accounts for 70–80% of the total cache. Reducing the on-GPU audio KV footprint is therefore a prerequisite for deploying long-form speech LLMs under realistic memory constraints and enables larger batch sizes on a fixed GPU budget.

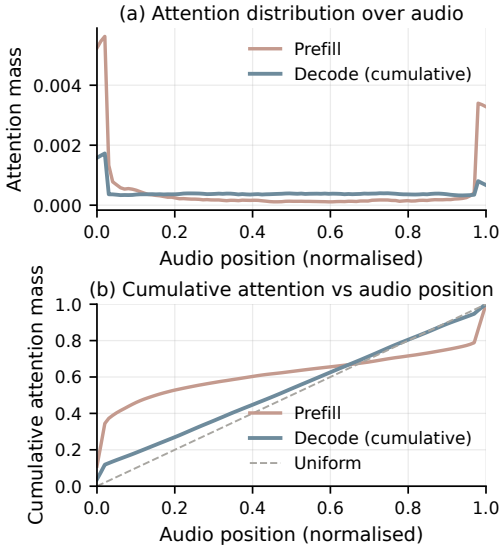


Figure 1: Comparison of attention weights distribution on audio positions for both *Prefill* and *Decode* phases, averaged across 50 samples from the LibriSpeech-Long `dev-clean` subset (Voxtral-Mini-3B). *Prefill*: from the first answer token; *Decode (cumulative)*: averaged over all answer tokens. (a) Per-position mass on an equal-width grid; (b) cumulative mass, with the diagonal marking a uniform distribution. Prefill mass concentrates near the start (sink effect); decode-cumulative mass spreads across the full input.

KV cache compression methods for text LLMs,

[*]Yiming Yao and Chenyang Lyu contributed equally. Work was done when Yiming Yao was interning at Alibaba.
[†]Jinsong Su is the corresponding author.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 24735–24748
October 24-29, 2026 ©2026 Association for Computational Linguistics

such as H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), and Ada-KV (Feng et al., 2025), score positions using attention from the prompt and permanently discard the low-ranked ones. AudioKV (Wang et al., 2026) adapts this paradigm to speech LLMs with FFT-smoothed scoring and an audio-tuned per-head budget. These methods share a common premise: that prefill attention reliably predicts which positions decoding will attend to.

We find that this premise does not hold on long-form audio. On LibriSpeech-Long with Voxtral-Mini-3B, prefill attention from the first answer token concentrates near the start of the audio (an attention-sink effect (Xiao et al., 2024)), while the attention accumulated over the full decode distributes its mass broadly across the clip; the top positions selected by the two signals overlap weakly (Figure 1; §2.1). Any method that commits to a retention set early in inference and lacks a pathway to recover discarded positions is vulnerable to this gap; the failure is most visible for irreversible baselines: at tight budgets they fail to terminate generation and produce word error rates well above 100%. Text-LLM methods that do add a recovery pathway (ArkVale (Chen et al., 2024), ClusterKV (Liu et al., 2025b)) rank KV by per-step query similarity rather than audio structure, and ArkVale's full CPU mirror leaves the total footprint unchanged.

To address the aforementioned challenges, we propose **WnW (Waxing-and-Waning KV cache)**,[1] which reduces the on-GPU audio KV footprint while keeping evicted positions recallable from CPU. WnW classifies KV heads into three functional classes via an offline calibration: a small set of *anchor* heads is kept fully on GPU and serves as a decode-time observer of audio importance; *tidal* heads keep a portion on GPU and the complement on CPU, available for recall; *fixed* heads keep only an on-GPU subset, with the rest permanently discarded. During decoding, anchor-head attention at each step is aggregated into chunk-level scores; the complements of the highest-scoring chunks are recalled from CPU for the tidal heads, while previously selected chunks that lose relevance are dropped from GPU. Deferring retention to decode time lets WnW correct prefill-time mistakes. The **key contributions** of this paper are:

- We quantify the prefill/decode attention mismatch on long-form audio using positional mass concentration and top-K Jaccard overlap, showing that any compression scheme without a recovery pathway is structurally limited.
- We design WnW around two mechanisms absent from text-LLM compression. The first is a head triage that scores each KV head by the product of its audio-grounding hit ratio and its gradient-based KV sensitivity, so the recallable capacity is reserved for the heads where discarding audio KV most damages quality. The second is a decode-time chunk swap driven by the anchor heads, which are selected by the same criterion and whose per-step attention tracks the generation frontier through the time-ordered audio, yielding a recall signal aligned to the underlying transcript.
- On LibriSpeech-Long with two 3B backbones (Voxtral-Mini-3B and Qwen2.5-Omni-3B), WnW remains within ~ 1.6 WER points of Full Cache even when only 20% of audio tokens remain on GPU, at which point irreversible baselines fail to terminate. Results generalize across languages, tasks, and domains; WnW scales to a 24B backbone, where it outperforms every baseline at **20%** retention; and CPU-GPU recall adds negligible decode-time overhead.

2 Method

Notation. Throughout, L denotes the number of decoder layers, H_q and H_{kv} are the per-layer query and key/value heads (with GQA group size $g{=}H_q/H_{\text{kv}}$ (Ainslie et al., 2023)), r_{tok} is the audio frame rate in tokens per second, and T_a is the number of audio KV positions in the input. For Voxtral-Mini-3B-2507 (Liu et al., 2025a) we have $L{=}30$, $H_q{=}32$, $H_{\text{kv}}{=}8$, $g{=}4$, and $r_{\text{tok}}{=}12.5$; for Qwen2.5-Omni-3B (Xu et al., 2025) we have $L{=}36$, $H_q{=}16$, $H_{\text{kv}}{=}2$, $g{=}8$, and $r_{\text{tok}}{=}25$.

Method Overview. WnW compresses the audio KV cache through two coordinated mechanisms: an *offline head triage* that partitions KV heads into three functional roles, and an *online recallable chunk swap* that tracks the generation frontier (Figure 2). The triage scores every KV head on a held-out set along two complementary axes, voice score (VS) and head sensitivity (HS), then assigns each head to one of three roles: **anchor heads** keep all audio KV on GPU, and their attention indicates which audio region the current token attends to; **tidal heads** keep a portion on GPU and offload the complement to CPU, recalling that complement on

[1] Code, experiment scripts, configurations, and dataset instructions will be released at https://github.com/XMUDeepLIT/WnW.

demand under anchor guidance; **fixed heads** keep a static per-segment prefill-time top-k on GPU with no recall pathway.

2.1 Prefill–Decode Attention Mismatch

Methods that compress the audio KV cache during prefill, both those built for text LLMs (SnapKV (Li et al., 2024), Ada-KV (Feng et al., 2025)) and those built for speech LLMs (AudioKV (Wang et al., 2026)), score each audio position once and keep the top-ranked ones, using attention from the prompt (or its last w tokens) as a proxy for decode-time importance. This pipeline assumes prefill attention predicts decode-time attention. Text-domain work has tried to sharpen the prefill estimate via pseudo-query generation (Wang et al., 2025b) and learned future-attention adapters (Ahn et al., 2026), acknowledging the proxy is imperfect. This section measures the gap directly on long-form audio.

Figure 1 compares two attention distributions over audio positions, averaged across 50 samples from the LibriSpeech-Long `dev-clean` subset (Voxtral-Mini-3B). The *prefill* distribution is the attention from the first answer token to each audio position, averaged over layers and heads; this is the signal prefill-scoring baselines use for compression decisions. The *decode-cumulative* distribution additionally averages over all answer tokens produced during decoding. Two findings emerge:

- **Attention sink dominates prefill (Xiao et al., 2024).** The prefill distribution places 47.9% of its mass on the first 10% of audio positions and 69.3% on the first half. The decode-cumulative distribution places only 9.8% on the first 10% and is close to uniform (47.7% / 52.3% on the two halves).
- **Top-K rankings overlap weakly.** The Jaccard overlap between top-K audio positions ranked by prefill and decode-cumulative attention is 0.187 at $K{=}100$ and 0.240 at $K{=}500$. This is well above the random baseline $\approx K/T_a$ (0.013 at $K{=}100$), but far from the agreement that would justify reusing prefill rankings during decoding.

Under these distributions, any method that fixes its retention set before decoding and lacks a recovery pathway will discard a substantial fraction of the positions decoding will query, irrespective of how that decision is scored. Across the 240 KV heads, per-head Jaccard ($K{=}100$) ranges from 0.006 to 0.641 (27% below 0.1): the mismatch is pervasive, but its quality impact is not, because heads also differ in how much the answer-token loss depends on their audio KV. The two factors compound: heads that both shift their attention as decoding progresses *and* carry loss-critical audio KV need retention that evolves with the generated text; heads that fail either condition tolerate prefill's coarse approximation. Specifically, without audio grounding, a head's top positions do not follow the current word through the clip, so the set that prefill picks stays close to what decoding attends to. Similarly, without loss sensitivity, quality is largely unaffected by which positions survive. WnW exploits this asymmetry: §2.2 identifies the quality-critical heads via offline calibration, and §2.3 invests recall overhead only in them.

Speech affords a structural advantage for decode-time correction: audio tokens are strictly time-ordered and word-aligned to the underlying transcript, so attention sweeps progressively through the audio as decoding advances. AudioKV (Wang et al., 2026) exploits this alignment during prefill to decide *how much* to retain per head (a static budget), but leaves the *which-positions* decision fixed. WnW takes the complementary step: it uses the temporal structure to decide *when* to recall, letting anchor-head attention track the generation frontier and fetching the relevant audio chunk on demand.

2.2 Head Functional Triage via Offline Calibration

In sequence-to-sequence learning, encoder layers differ sharply in importance, and generation quality hinges on a small subset that the decoder preferentially attends to (Liu et al., 2021). Audio KV heads exhibit the same asymmetry, so we score each KV head on a held-out set with two complementary signals: a *voice score* (VS) measuring how strongly its attention is grounded in audio content, and a *head sensitivity* (HS) measuring how much the answer-token loss depends on its audio KV. We combine them into a modality-aware importance score $\mathrm{VS}_{l,k} \times \mathrm{HS}_{l,k}$ that drives all triage decisions.

Voice Score (VS). We measure each head's *audio-content grounding* as a hit ratio: the fraction of its top-attended audio positions that fall inside the word-aligned time span of the current answer token, with the alignment taken from WhisperX (Radford et al., 2023; Bain et al., 2023). This construction follows AudioKV (Wang et al., 2026), itself adapted from SparseMM (Wang et al., 2025a). We average the ratio over answer tokens and cal-

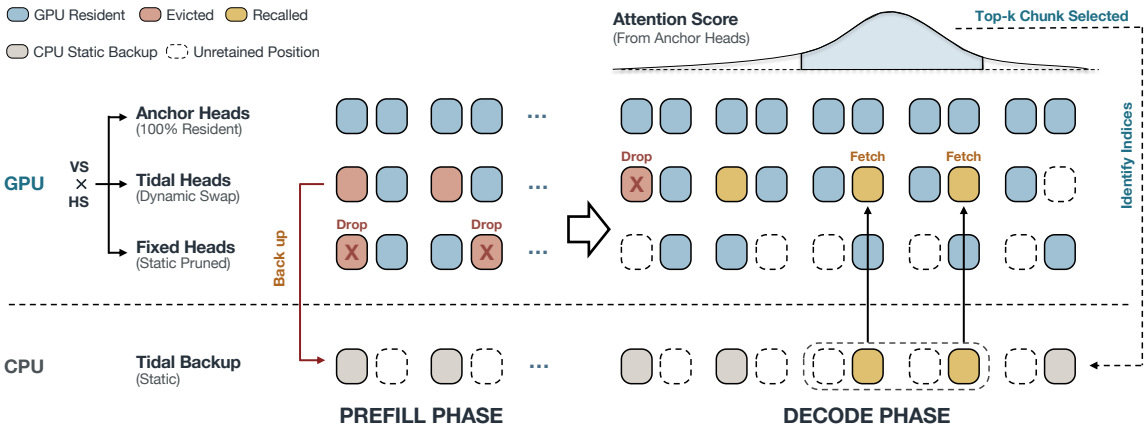


Figure 2: Overview of WnW. **Left (Prefill)**: KV heads are partitioned into three roles via offline VS×HS calibration. Anchor heads retain all audio KV on GPU; tidal heads offload their unselected complement to CPU; fixed heads discard the unselected portion permanently. **Right (Decode)**: anchor-head attention identifies the top-k audio chunks at each step; tidal heads recall those chunks' CPU-side complement on demand and release it again after δ consecutive unselected steps (the CPU copy persists). The dashed frame marks one selected chunk, i.e. two segments.

ibration samples, then over the query heads that share each KV head, giving a per-KV-head voice score $\mathrm{VS}_{l,k}$ (details in Appendix E).

Head Sensitivity (HS). Voice score captures attention patterns but not loss impact. We complement it with a gradient-based KV importance (Molchanov et al., 2017; Michel et al., 2019): $\mathrm{HS}_{l,k}$ is the ℓ_2-norm of the answer-token loss gradient with respect to each head's audio key vectors, averaged over positions, tokens, and calibration samples (Appendix E).

Three-Way Classification. We use the VS×HS ranking over all $L \times H_{\text{kv}}$ KV heads in two nested steps. First, the top n_{voice} heads form the **voice heads**: those whose attention is grounded in audio content *and* whose audio KV is loss-critical; their decode-time attention shifts as different words are generated, so they benefit from a recall pathway. The remaining heads are **fixed heads**: their audio KV is either not audio-grounded or not loss-critical, so we retain a static per-segment top-k subset on GPU without recall. Second, within the voice heads, the top n_{anchor}=5 are designated **anchor heads** and keep their full audio KV on GPU: as the most audio-grounded and loss-critical heads, they provide the most reliable per-step signal of which audio chunks matter, which we reuse to drive chunk swap (§2.3) for the tidal heads. The remaining voice heads are **tidal heads** and keep a portion on GPU with the complement on CPU.

Tidal and fixed heads share a proportional *audio-KV* retention schedule:

$$\text{retention}_i = \min(\tilde{s}_i \cdot \lambda,\ 1.0), \tag{1}$$

where retention_i is the fraction of *audio* positions kept on GPU for head i (non-audio KV is always retained in full), and $\tilde{s}_i \in [0, 1]$ is its normalized importance score. WnW exposes two hyperparameters: the voice-head count n_{voice} controls how many heads have CPU-backed recallable storage, and the scale λ controls the per-head audio retention magnitude. When λ is large, all voice heads clip to retention 1 and the recallable CPU partition is dormant; recall activates only when the GPU budget is tight enough to push λ out of saturation.

2.3 Prefill Compression and Decode-Time Chunk Swap

Chunk Structure. We partition the T_a audio positions into overlapping *chunks* of W_c=4 s with stride W_s=2 s. A chunk holds $n_c = \lceil W_c \cdot r_{\text{tok}} \rceil$ audio tokens and spans two contiguous *segments* of $n_s = \lceil W_s \cdot r_{\text{tok}} \rceil$ tokens, so adjacent chunks share a segment; load and eviction operate at the segment granularity. With r_{tok}=12.5 for Voxtral-Mini-3B, this yields n_c=50 and n_s=25; with r_{tok}=25 for Qwen2.5-Omni-3B, n_c=100 and n_s=50.

Prefill Compression. During prefill, each non-anchor KV head (l, k) ranks audio positions by aggregated text-to-audio attention $p^{\text{prefill}}_{l,k,\tau} = \sum_{t \in \text{text}} \text{attn}_{l,k,t,\tau}$. Selection is performed *per segment*: within each n_s-token segment, the top $\max(1, \lfloor \text{retention}_{l,k} \cdot n_s \rfloor)$ positions are kept. This

guarantees uniform temporal coverage, since no segment is wholly evicted, providing a stable initial state for chunk swap; we ablate it against a global top-k variant in §3.8. For tidal heads, unselected positions are offloaded to CPU and remain recallable; for fixed heads, they are permanently discarded. Anchor heads keep all audio positions on GPU.

Decode-Step Score Aggregation. At every decode step s, after attention is computed for all L layers, the anchor Q-heads' softmax weights at audio positions are aggregated across anchor (layer, q-head) pairs into a per-position score:

$$A_\tau^{(s)} = \sum_{(l,h)\in\text{anchor}} \text{softmax}\left(\frac{q_{l,h}^{(s)} K_{l,h}^\top}{\sqrt{d}}\right)_\tau, \quad \tau \in \text{audio}. \tag{2}$$

The accumulator is reset to zero at the end of each step, so $A_\tau^{(s)}$ reflects only the current decode step, not a running sum across steps. This lets the importance signal track the generation frontier rather than being dominated by audio regions that mattered earlier.

Dynamic Chunk Selection. After each decode step, chunk c receives score $\bar{A}_c^{(s)} = \text{mean}_{\tau\in c} A_\tau^{(s)}$; the top-$k$ (k=3) chunks are selected. For tidal heads, the positions selected during prefill remain GPU-resident throughout decoding, and what moves is their CPU-side complement: for a selected chunk, the complement of its segments is fetched from CPU onto GPU; a fetched complement left unselected for δ=3 consecutive steps is dropped from GPU, while its CPU copy persists and can be re-fetched if the chunk regains importance.[2] The eviction lag δ buffers short-term score fluctuations and prevents thrashing.

3 Experiments

3.1 Models

The main backbone is **Voxtral-Mini-3B-2507** (Liu et al., 2025a), a 3B-parameter audio–language model; for cross-model evaluation we additionally use **Qwen2.5-Omni-3B** (Xu et al., 2025) (per-model constants in §2), whose feature extractor uses a single 300-second window (in contrast to Voxtral's concatenated 30-second windows); all clips evaluated on it fit within that window. Appendix B reports a larger-scale check on **Voxtral-Small-24B-2507**. All inference uses bf16 weights, greedy decoding, and the default chat template.

[2]The name *WnW* (waxing and waning) refers to chunks moving across the GPU/CPU boundary as their per-step anchor-head attention rises and falls.

3.2 Datasets

LibriSpeech-Long (Park et al., 2025) is a long-form benchmark constructed from LibriSpeech (Panayotov et al., 2015) by merging adjacent utterances into recordings of up to four minutes. We use `test-clean` and `test-other` (270 and 207 samples) as the main English ASR benchmark.

LongSpeech (Yang et al., 2026) is a multilingual long-form speech-LLM benchmark. We use its French ASR split (`asr-fr`) and English-to-French speech translation split (`en2fr`), sub-sampling 200 clips from each (fixed seed).

PriMock57 (Papadopoulos Korfiatis et al., 2022) is a set of 57 simulated primary-care consultations, which we use as an out-of-domain ASR benchmark with its manual transcripts as references.

Calibration set. 50 LibriSpeech-Long `dev-clean` samples, disjoint from all test sets.

3.3 Baselines

We compare against five baselines, all using their authors' recommended hyperparameters unless noted (Appendix F):

- **Full Cache** — no compression, the quality upper bound.
- **Ada-KV** (Feng et al., 2025) — adaptive per-head budget allocation derived from a theoretical loss upper bound.
- **AudioKV** (Wang et al., 2026) — voice-score-guided per-head budget with FFT smoothing.
- **ArkVale** (Chen et al., 2024) — page-based recallable KV eviction with bounding-volume page summaries.
- **AffPool** (Xiang et al., 2026) — a token-merging efficiency baseline that pools adjacent audio tokens by affinity during prefill.

Audio-only compression constraint (modified baselines). We restrict all KV-management baselines to the audio-token region of the KV cache; text tokens (system prompt, user prompt, and tokens generated during decoding) are kept in full. The original implementations of Ada-KV, AudioKV, and ArkVale operate on the entire sequence, which would mix audio compression with prompt and generated-token compression; restricting them

to the audio region matches WnW's compression scope. AffPool needs no such restriction, since it operates only on audio tokens.

The baselines span multiple axes: management paradigm (irreversible vs. recallable), domain (text vs. audio), and compression target (KV eviction vs. token merging). Each represents a broader family.[3] AffPool stands apart: it merges audio tokens before the KV cache exists, so it shrinks prefill computation as well as the cache, and its budget is the remaining audio-token ratio rather than a retention set over original positions.

3.4 Audio KV Retention and WnW Configuration

Audio KV retention. Given the audio-only compression scope above, each method's compression budget is naturally expressed on the audio region. Let $N_{\text{full}}^{\text{audio}} = L \cdot H_{\text{kv}} \cdot T_a$ be the total number of audio KV positions across all (layer, KV head) pairs:

$$r_{\text{GPU}} = \frac{N_{\text{GPU}}^{\text{audio}}}{N_{\text{full}}^{\text{audio}}}, \quad r_{\text{GPU+CPU}} = \frac{N_{\text{GPU}}^{\text{audio}} + N_{\text{CPU}}^{\text{audio}}}{N_{\text{full}}^{\text{audio}}},$$

where $N_{\text{GPU}}^{\text{audio}}$ and $N_{\text{CPU}}^{\text{audio}}$ are the audio KV token counts kept on device and (recallably) on host, respectively. r_{GPU} is the on-device footprint that determines GPU memory; $r_{\text{GPU+CPU}}$ additionally counts any recallable CPU-resident audio KV. For methods without a CPU-resident complement (Ada-KV, AudioKV, Full Cache) the two coincide; for WnW (tidal heads) and ArkVale (evicted pages) they may differ.

Configuration. We evaluate each method at four target retention levels $r_{\text{GPU}} \in \{0.8, 0.6, 0.4, 0.2\}$. Ada-KV, AudioKV, and ArkVale expose a single hyperparameter that maps directly to per-head GPU usage. For WnW, we select λ on the calibration set so that the measured r_{GPU} matches each target, using $n_{\text{voice}} \in \{30, 30, 50, 90\}$ on Voxtral-Mini-3B ($L \times H_{\text{kv}}$=240) and $\{9, 9, 15, 27\}$ on Qwen2.5-Omni-3B ($L \times H_{\text{kv}}$=72) for $r_{\text{GPU}} \in \{0.8, 0.6, 0.4, 0.2\}$ respectively. $r_{\text{GPU+CPU}}$ equals r_{GPU} down to 0.4 and diverges only at 0.2, where the recall pathway activates and it reaches ≈ 0.42 (Table 5).

[3] Ada-KV represents SnapKV (Li et al., 2024), PyramidKV (Cai et al., 2025), HeadKV (Fu et al., 2025), RazorAttention (Tang et al., 2025), and ChunkKV (Liu et al., 2025c): whatever rule allocates the budget, all of them fix the retention set during prefill with no recovery pathway, which is the property §2.1 tests. ArkVale represents Quest (Tang et al., 2024), ClusterKV (Liu et al., 2025b), and InfiniGen (Lee et al., 2024). KV quantization (Liu et al., 2024) is orthogonal and compatible with WnW.

3.5 Metrics

Word Error Rate. We report *truncated WER* after light text normalization (lower-casing and removal of all punctuation except apostrophe and period): each hypothesis is first truncated to 1.2× the ground-truth token length and then scored against the reference. Non-terminating hypotheses are common for irreversible baselines at low budgets; truncation prevents them from inflating insertion errors arbitrarily and isolates the effect of audio compression on the transcribed prefix. Substitutions/deletions/insertions are computed with the `jiwer` library.

Audio KV Retention. For each method we report the *measured* on-device retention r_{GPU} (defined in §3.4), since the realized value may differ from the target by a few points. Tables 2 and 5 add WnW's combined retention $r_{\text{GPU+CPU}}$.

3.6 Main Comparison

We compare WnW against the baselines on LibriSpeech-Long across the four target audio KV retention levels r_{GPU}, on both the Voxtral-Mini-3B and Qwen2.5-Omni-3B backbones (Figure 3; full numbers in Appendix Table 5).

Across both backbones, WnW stays close to the Full Cache at every retention level r_{GPU}, within ∼1.6 WER on Qwen2.5-Omni-3B. The baselines fall into a consistent ordering as r_{GPU} decreases (in WER, lower is better): Ada-KV ≫ AudioKV > ArkVale > WnW. This ordering matches the prefill–decode mismatch analysis in §2.1: irreversible early compression (Ada-KV, AudioKV) inherits a structural error that grows with the eviction ratio, and both fail to terminate at tight r_{GPU}; adding a recall pathway (ArkVale) prevents non-termination but is insufficient alone; combining recall with audio-aware long-horizon scoring from anchor heads (WnW) closes the remaining gap. Repetition is the documented form of this failure: Wang et al. (2026) observe "catastrophic performance degradation and repetition" when text-LLM eviction is applied to audio. On Qwen2.5-Omni-3B, ArkVale slightly exceeds WnW at high retention: it keeps a complete CPU mirror of evicted pages ($r_{\text{GPU+CPU}}$=100%), so r_{GPU} controls only how much KV is GPU-resident, not how much

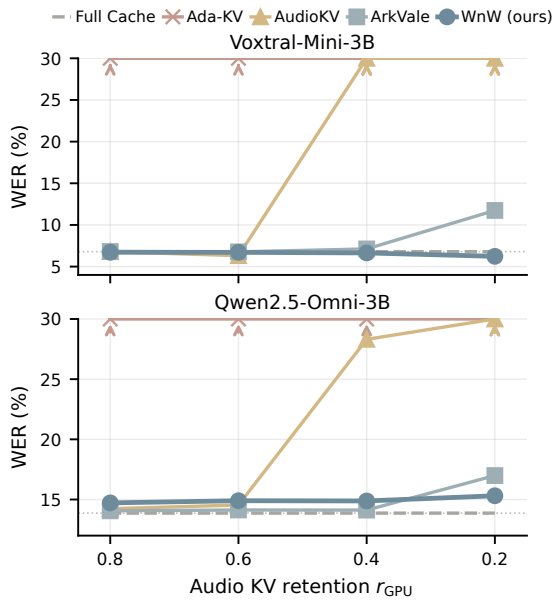


Figure 3: Truncated WER vs. audio KV retention r_{GPU} on LibriSpeech-Long `test-clean`. Top: Voxtral-Mini-3B; bottom: Qwen2.5-Omni-3B. Upward arrows mark points exceeding 30% WER (numbers in Appendix Table 5). WnW stays close to Full Cache (dashed) at every r_{GPU}; irreversible baselines degrade sharply below r_{GPU}=0.6.

audio stays recoverable, whereas WnW grants CPU recall only to tidal heads and permanently discards the unselected audio KV of fixed heads ($r_{\text{GPU+CPU}}\approx$80% at r_{GPU}=0.8). This is the intended storage–accuracy tradeoff of head triage: WnW stores less and is substantially better at low retention, where precise anchor-guided recall matters most. On Voxtral-Mini-3B, WnW tracks the Full Cache within $\leq$ 0.6 WER at every r_{GPU}, always marginally below it except on `test-other` at r_{GPU}=0.2; this gap is within the variation expected from greedy decoding and our truncation rule, not a real quality difference.[4]

Table 1 compares WnW with AffPool. AffPool recovers near-full-cache quality only at high retention, but collapses at 20%–40% retention. This suggests that prefill-time token merging shares the core limitation of the irreversible baselines: it commits to its decision before decoding begins and leaves no way back. Merging adds an error-propagation effect on top, since merges made in early layers affect all subsequent layers. WnW avoids this failure mode by preserving token identities and deferring recall decisions to decode time. The KV-management ordering above also holds on the larger Voxtral-Small-24B backbone, where WnW remains the best at ∼20% retention while the irreversible baselines still collapse (Appendix B).

[4]Our Qwen2.5-Omni-3B AudioKV result at r_{GPU}=0.4 is higher than reported by Wang et al. (2026). The official implementation is not public, so the gap may reflect implementation and evaluation differences, including head granularity under GQA, prompt template, `max_new_tokens`, and WER scoring. We apply one pipeline consistently to every method.

Method	Category	Retention	clean / other
Full Cache	—	100%	**6.79 / 8.86**
WnW	KV recall	∼20%	**6.23 / 8.87**
AffPool	t.m.	20%	111.63 / 113.25
AffPool	t.m.	40%	67.08 / 75.14
AffPool	t.m.	60%	10.04 / 11.00
AffPool	t.m.	80%	6.47 / 8.94

Table 1: Comparison with a token-merging (t.m.) efficiency baseline on LibriSpeech-Long using Voxtral-Mini-3B. Numbers are truncated WER (↓, ratio 1.2).

Method	r_{GPU}	LongSpeech		PriMock57
		asr-fr↓	en2fr↑	WER↓
Full Cache	100%	**20.42**	**38.48**	**23.47**
Ada-KV	∼20%	112.91	2.99	106.43
AudioKV	∼20%	105.00	5.22	103.83
ArkVale	∼20%	28.63	35.40	32.45
WnW (ours)	∼20%	**22.68**	**38.21**	**24.23**

Table 2: Generalization and domain robustness on Voxtral-Mini-3B at r_{GPU}=0.2 (200 samples per LongSpeech split, all 57 PriMock57 consultations, the same samples for every method). asr-fr and PriMock57: truncated WER (ratio 1.2); en2fr: sacreBLEU (Post, 2018), same truncation. WnW's measured retention is r_{GPU}=22.25% / $r_{\text{GPU+CPU}}$=41.5% on LongSpeech and 21.2% / 40.46% on PriMock57.

3.7 Generalization and Domain Robustness

To test whether the WnW design transfers beyond English ASR, we evaluate at the most aggressive setting r_{GPU}=0.2 on two LongSpeech splits, `asr-fr` (French ASR) and `en2fr` (English→French speech translation), which together probe two orthogonal axes of generalization: cross-lingual and cross-task. We further test domain robustness on PriMock57 medical consultations (Table 2).

WnW transfers to a different language, a different task, and a different domain: relative to Full Cache it loses 2.3 WER points on French ASR, 0.27 BLEU on translation, and 0.76 WER points on medical dialogue. Ada-KV and AudioKV again fail to terminate, with hypotheses far longer than

the reference and BLEU near zero. ArkVale avoids non-termination but trails WnW throughout, indicating that recall alone, without head triage, is insufficient. That the LibriSpeech-calibrated roles hold up under all three shifts suggests the partition is not tightly overfit to the calibration distribution. In a matched re-calibration study, replacing the English LibriSpeech calibration set with French LongSpeech changes English WER by at most 0.04 points and French WER by 1.12 points, with $76/85$ tidal heads preserved (Appendix D).

3.8 Ablation Studies

We run all ablations on LibriSpeech-Long `test-clean` with truncated WER (ratio 1.2). A1 and A2 are reported at both r_{GPU}=0.1 and 0.2, since their effect depends on whether the recall pathway is active.

(A1) Voice-head count n_{voice}. We sweep $n_{\text{voice}} \in \{10, 30, 50, 70, 90\}$ at both retention levels, retuning λ per cell so the on-GPU footprint matches the target. Since anchor heads stay at full retention regardless of n_{voice}, any quality change reflects the recall pathway: larger n_{voice} converts fixed heads into CPU-backed tidal heads without enlarging the on-GPU budget.

(A2) Prefill selection granularity. We compare the default per-segment top-k compression (§2.3) against a global top-k variant that spends the same total budget by ranking all audio positions of a head together with no per-segment quota. They share identical head triage, retention, and decode-time chunk swap, differing only in how prefill picks which positions to keep on GPU. We report at `nv30`/r_{GPU}=0.1 and `nv90`/r_{GPU}=0.2.

(A3) Head triage signal. A single score drives both selections, the top n_{voice} voice heads and the top 5 anchors, so we compare VS alone, HS alone, and VS $\times$ HS (WnW), holding everything else fixed at `nv30`/r_{GPU}=0.1.

A1 (Table 3) shows that larger tidal pools matter most under aggressive compression: at r_{GPU}=0.1, increasing n_{voice} from 10 to 70 reduces WER from 51.92% to 6.31%. At r_{GPU}=0.2 the same range moves WER by less than half a point, since the wider GPU budget already keeps most of what each head needs on device. A2 shows that per-segment prefill selection matters when r_{GPU} is tight: at r_{GPU}=0.1, dropping the per-segment quota costs 3.33 WER points, since global top-k lets entire segments be evicted during prefill, leaving chunk swap no GPU-side starting point there; the gap closes to 0.04 at r_{GPU}=0.2 when retention is high enough that uniform coverage is preserved either way. A3 shows that the two signals are not interchangeable under tight r_{GPU}: HS alone trails VS $\times$ HS by 26.63 WER points and VS alone fails to terminate (124.66%). Voice score by itself can promote audio-grounded heads whose KV is not loss-critical, while sensitivity alone can include heads that ignore the audio; only the product satisfies both criteria. Appendix C further shows that WnW is insensitive to chunk size and anchor-head count around the default configuration.

(A1) Voice-head count (WER ↓)		
n_{voice}	r_{GPU}=0.1	r_{GPU}=0.2
10	51.92	6.53
30	9.78	6.46
50	8.09	6.39
70	**6.31**	**6.20**
90	6.76	6.23
(A2) Prefill selection (WER ↓)		
	`nv30`/0.1	`nv90`/0.2
per-segment (ours)	**9.78**	**6.23**
global top-k	13.11	6.27
(A3) Triage signal at `nv30`/r_{GPU}=0.1 *(WER ↓)*		
VS $\times$ HS (ours)	**9.78**	
HS only	36.41	
VS only	124.66	

Table 3: Ablations on LibriSpeech-Long `test-clean` (truncated WER, ratio 1.2).

3.9 Efficiency and Runtime Analysis

Figure 4 shows the on-GPU KV cache size over decode steps under four compression ratios. All curves rise linearly because, per the audio-only constraint (§3.3), every method keeps generated tokens in full. At the same target r_{GPU}, all methods achieve nearly identical audio-KV footprints, so the WER differences in Figure 3 reflect how each method uses its budget, not how much GPU memory it consumes. Within this fixed GPU budget, WnW delivers near-full-cache accuracy down to r_{GPU}=0.2.

Table 4 isolates recall cost by varying the number of tidal heads, the only heads that transfer from CPU. Transfer grows linearly with tidal-head count but stays small ($\leq$ 1.04 MB/step), and median decode time varies by less than 5% across a $6.7\times$ range of tidal heads. CPU-GPU recall is thus not

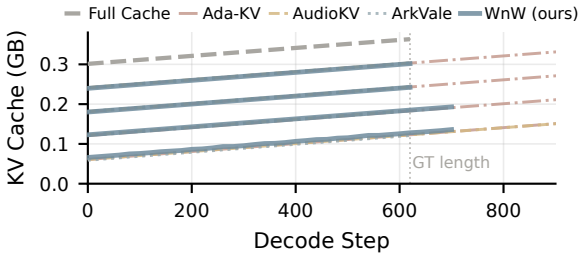


Figure 4: GPU KV cache size across decoding steps for a 4-minute LibriSpeech-Long sample (Voxtral-Mini-3B). Each method is evaluated with $r_{\text{GPU}} \in \{0.8, 0.6, 0.4, 0.2\}$, yielding four curves. The dotted vertical line denotes the ground-truth sequence length; methods whose curves extend beyond it fail to terminate. The slight oscillation in WnW reflects chunk swapping.

Config.	r_{GPU}	ms/token	MB/step
35 tidal heads	21.7%	27.2	0.05
75 tidal heads	21.4%	26.8	0.20
115 tidal heads	21.0%	26.7	0.40
155 tidal heads	20.5%	27.4	0.62
195 tidal heads	20.1%	27.5	0.82
235 tidal heads	19.7%	27.8	1.04

Table 4: Runtime impact of CPU→GPU recall on Voxtral-Mini-3B, LibriSpeech-Long `test-clean` (64 samples) at ∼20% GPU retention, using one A100 with batch size 1. Decode time is median ms/token; MB/step is CPU→GPU KV transfer averaged per decode step.

the dominant bottleneck here.

4 Related Work

Irreversible KV Cache Compression. Scissorhands (Liu et al., 2023), H2O (Zhang et al., 2023), and SnapKV (Li et al., 2024) retain a top-ranked subset of positions chosen from prefill or accumulated attention, and evicted positions are never re-admitted. Ada-KV (Feng et al., 2025), PyramidKV (Cai et al., 2025), and ChunkKV (Liu et al., 2025c) refine per-layer, per-head, or per-chunk allocation under the same one-way eviction. All target text LLMs; §2.1 shows their prefill-attention premise fails on long-form audio.

Head-Level KV Management. Some text-LLM methods allocate cache per head rather than per position. HeadKV (Fu et al., 2025) scores heads for retrieval and reasoning ability to set per-head budgets, and RazorAttention (Tang et al., 2025) separates retrieval from non-retrieval heads, protecting the former in full while compressing the latter into a coarse compensation summary. WnW differs in both basis and consequence. The basis is audio grounding combined with gradient-based KV sensitivity rather than text-retrieval behavior, so head roles reflect how much discarding a head's *audio* KV hurts quality. The consequence is that each head receives a storage tier rather than a cache budget: a tidal head keeps an exact copy of its unselected audio KV on CPU and can take it back unchanged mid-decode, whereas KV compressed into a summary is unrecoverable. Anchor heads further act as a decode-time importance observer, absent from irreversible head-level schemes.

CPU-GPU KV Management. Building on paged GPU KV management (Kwon et al., 2023), ArkVale (Chen et al., 2024) and ClusterKV (Liu et al., 2025b) evict KV pages or clusters and recall them on demand via bounding-volume or semantic digests; Quest (Tang et al., 2024) performs query-aware selective loading per decode step; InfiniGen (Lee et al., 2024) speculatively prefetches KV for upcoming steps. All target text LLMs and rely on per-step query similarity. WnW operates on time-aligned audio chunks, splits heads into functional roles with anchors as a dedicated importance observer, and aggregates attention across all anchor (layer, Q-head) pairs per decode step rather than scoring one query against page summaries.

Speech LLM KV Compression. AudioKV (Wang et al., 2026) adapts SnapKV-style static eviction to speech LLMs with FFT-smoothed scoring and an audio-tuned per-head budget; our voice score reuses its head-scoring hit ratio, itself adapted from SparseMM's visual score (Wang et al., 2025a). WnW combines it with gradient-based head sensitivity (Molchanov et al., 2017; Michel et al., 2019), and drives a recallable decode-time chunk swap rather than static prefill budgets. To our knowledge, this is the first recallable, decode-driven KV management for speech LLMs.

5 Conclusion

WnW addresses the prefill–decode attention mismatch on long-form audio by deferring retention decisions to decode time: an offline triage assigns anchor, tidal, and fixed roles, and tidal heads recall audio chunks from CPU under anchor guidance. It stays close to Full Cache on 3B backbones at 20% GPU retention, generalizes across language, task, and domain, and outperforms KV-management baselines on 24B.

Limitations

The chunk-swap CPU-GPU transfers are issued synchronously; although our measurements show limited decode-time overhead, overlapping recall with attention via CUDA streams is a natural next optimization. The recall pathway is dormant in the high-r_{GPU} regime (§2.2); decoupling n_{voice} and λ to keep recall active when GPU memory is plentiful could yield further gains. Our evaluation targets offline decoding: extending WnW to streaming recognition is natural for speech-LLM architectures that append newly arrived audio to a shared, monotonically growing KV cache, as in recent LLM-based simultaneous speech translation systems that avoid re-encoding as input arrives (Ouyang et al., 2024). The mechanism does not, however, transfer to cross-attention (e.g., Whisper), transducer, or state-space architectures, which keep no length-dependent audio KV cache to manage. Applying WnW to non-transcription audio tasks (QA, dialogue, summarization) is promising once the underlying speech LLMs reach reliable quality on those tasks.

Acknowledgments

The project was supported by National Natural Science Foundation of China (No. 62676342), the Open Competition for Innovative Projects of Xiamen (No. 3502Z20251012), Natural Science Foundation of Fujian Province of China (No. 2024J011001), and the Public Technology Service Platform Project of Xiamen (No.3502Z20231043). We also thank the reviewers for their insightful comments.

References

Jinwoo Ahn, Ingyu Seong, Akhil Kedia, Junhan Kim, Hyemi Jang, Kangwook Lee, and Yongkweon Jeon. 2026. LookaheadKV: Fast and accurate KV cache eviction by glimpsing into the future without generation. In *Proceedings of ICLR 2026*.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of EMNLP 2023*.

Max Bain, Jaesung Huh, Tengda Han, and Andrew Zisserman. 2023. WhisperX: Time-accurate speech transcription of long-form audio. In *Proceedings of INTERSPEECH 2023*.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and 1 others. 2025. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. In *Proceedings of COLM 2025*.

Renze Chen, Zhuofeng Wang, Beiquan Cao, Tong Wu, Size Zheng, Xiuhong Li, Xuechao Wei, Shengen Yan, Meng Li, and Yun Liang. 2024. Arkvale: Efficient generative LLM inference with recallable key-value eviction. In *Proceedings of NeurIPS 2024*.

Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. 2025. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. In *Proceedings of NeurIPS 2025*.

Yu Fu, Zefan Cai, Abedelkadir Asi, Wayne Xiong, Yue Dong, and Wen Xiao. 2025. Not all heads matter: A head-level KV cache compression method with integrated retrieval and reasoning. In *Proceedings of ICLR 2025*.

Yan Gao, Yazheng Yang, Zhibin Lan, Yidong Chen, Min Zhang, Daimeng Wei, Derek F. Wong, and Jinsong Su. 2026. Towards fine-grained code-switch speech translation with semantic space alignment. In *Proceedings of IJCAI-ECAI 2026*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of SOSP 2023*.

Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *Proceedings of OSDI 2024*.

Yi Li, Rui Zhao, Ruiquan Zhang, Jinsong Su, Daimeng Wei, Min Zhang, and Yidong Chen. 2026. PLaST: Towards paralinguistic-aware speech translation. In *Proceedings of AAAI 2026*, pages 31805–31813.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. Snapkv: Llm knows what you are looking for before generation. In *Proceedings of NeurIPS 2024*.

Alexander H Liu, Andy Ehrenberg, Andy Lo, Clément Denoix, Corentin Barreau, Guillaume Lample, Jean-Malo Delignon, Khyathi Raghavi Chandu, Patrick von Platen, Pavankumar Reddy Muddireddy, and 1 others. 2025a. Voxtral. *Preprint*, arXiv:2507.13264.

Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. 2025b. Clusterkv: Manipulating llm kv cache in semantic space for recallable compression. In *Proceedings of DAC 2025*.

Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Yue Liu, Bo Li, Xuming Hu, and Xiaowen Chu. 2025c. Chunkkv: Semantic-preserving kv cache compression for efficient long-context llm inference. In *Proceedings of NeurIPS 2025*.

Xuebo Liu, Longyue Wang, Derek F. Wong, Liang Ding, Lidia S. Chao, and Zhaopeng Tu. 2021. Understanding and improving encoder layer fusion in sequence-to-sequence learning. In *Proceedings of ICLR 2021*.

Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. In *Proceedings of NeurIPS 2023*.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of ICML 2024*.

Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? In *Proceedings of NeurIPS 2019*.

Pavlo Molchanov, Stephen Tyree, Tero Karras, Timo Aila, and Jan Kautz. 2017. Pruning convolutional neural networks for resource efficient inference. In *ICLR*.

Siqi Ouyang, Xi Xu, Chinmay Dandekar, and Lei Li. 2024. FASST: Fast LLM-based simultaneous speech translation. *Preprint*, arXiv:2408.09430.

Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. 2015. LibriSpeech: An ASR corpus based on public domain audio books. In *Proceedings of ICASSP 2015*.

Alex Papadopoulos Korfiatis, Francesco Moramarco, Radmila Sarac, and Aleksandar Savkov. 2022. PriMock57: A dataset of primary care mock consultations. In *Proceedings of ACL 2022 (Short Papers)*.

Se Jin Park, Julian Salazar, Aren Jansen, Keisuke Kinoshita, Yong Man Ro, and RJ Skerry-Ryan. 2025. Long-form speech generation with spoken language models. In *Proceedings of ICML 2025*. Dataset: `https://github.com/google-deepmind/librispeech-long`.

Matt Post. 2018. A call for clarity in reporting BLEU scores. In *Proceedings of WMT 2018*.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust speech recognition via large-scale weak supervision. In *Proceedings of ICML 2023*.

Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Danning Ke, Shikuan Hong, Yiwu Yao, and Gongyi Wang. 2025. Razorattention: Efficient KV cache compression through retrieval heads. In *Proceedings of ICLR 2025*.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-aware sparsity for efficient long-context llm inference. In *Proceedings of ICML 2024*.

Jiahui Wang, Zuyan Liu, Yongming Rao, and Jiwen Lu. 2025a. SparseMM: Head sparsity emerges from visual concept responses in MLLMs. In *Proceedings of ICCV 2025*.

Yixuan Wang, Shiyu Ji, Yijun Liu, Yuzhuang Xu, Yang Xu, Qingfu Zhu, and Wanxiang Che. 2025b. Lookahead Q-cache: Achieving more consistent KV cache eviction via pseudo query. In *Proceedings of EMNLP 2025*.

Yuxuan Wang, Peize He, Xiyan Gui, Xiaoqian Liu, Junhao He, Xuyang Liu, Zichen Wen, Xuming Hu, and Linfeng Zhang. 2026. Audiokv: Kv cache eviction in efficient large audio language models. *Preprint*, arXiv:2604.06694.

Suhang Wu, Jialong Tang, Chengyi Yang, Pei Zhang, Baosong Yang, Junhui Li, Junfeng Yao, Min Zhang, and Jinsong Su. 2025. Locate-and-focus: Enhancing terminology translation in speech language models. In *Proceedings of ACL 2025*.

Bajian Xiang, Tingwei Guo, Xuan Chen, and Yang Han. 2026. Do we need distinct representations for every speech token? unveiling and exploiting redundancy in large speech language models. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 15069–15087.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *Proceedings of ICLR 2024*.

Jin Xu, Zhifang Guo, Jinzheng He, Hangrui Hu, Ting He, Shuai Bai, Keqin Chen, Jialin Wang, Yang Fan, Kai Dang, Bin Zhang, Xiong Wang, Yunfei Chu, and Junyang Lin. 2025. Qwen2.5-Omni technical report. *Preprint*, arXiv:2503.20215.

Fei Yang, Xuanfan Ni, Renyi Yang, Jiahui Geng, Qing Li, Chenyang Lyu, Yichao Du, Longyue Wang, Weihua Luo, and Kaifu Zhang. 2026. LongSpeech: A scalable benchmark for transcription, translation and understanding in long speech. In *Proceedings of ICASSP 2026*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Proceedings of NeurIPS 2023*.

A Full Main Results

Table 5 (next page) reports the full LibriSpeech-Long results behind Figure 3, including

`test-other` numbers and WnW's measured r_{GPU} / $r_{\text{GPU+CPU}}$ at every retention level on both backbones.

B Scaling to a Larger Speech LLM

We evaluate whether WnW's advantage over KV-management baselines persists beyond 3B-parameter backbones. Table 6 reports results on Voxtral-Small-24B at ∼20% GPU retention. Head roles are recalibrated for the 24B model, while chunk size and anchor-head count are transferred from Voxtral-Mini-3B. In the notation of §2, this backbone has L=40, H_q=32, H_{kv}=8, g=4, a head dimension of 128, and the same r_{tok}=12.5 as Voxtral-Mini-3B, so triage runs over $L \times H_{\text{kv}}$=320 KV heads instead of 240.

Method	r_{GPU}	**test-clean**	**test-other**
Full Cache	100%	**5.70**	**8.37**
WnW (ours)	∼20%	**11.29**	**16.63**
ArkVale	∼20%	15.60	22.20
Ada-KV	∼20%	110.51	113.05
AudioKV	∼20%	95.02	98.74

Table 6: Scaling to Voxtral-Small-24B on LibriSpeech-Long. Numbers are truncated WER (↓, ratio 1.2).

WnW remains the best KV-management method at ∼20% GPU retention on the 24B backbone. The absolute gap to Full Cache is larger than on 3B models, reflecting the stronger base model and lower Full Cache WER, but irreversible baselines collapse and ArkVale remains substantially worse than WnW.

C Hyperparameter Sensitivity

Table 7 reports sensitivity sweeps on Voxtral-Mini-3B at ∼20% GPU retention on LibriSpeech-Long `test-clean`. WnW is stable across a broad range of chunk sizes and anchor-head counts.

Chunk size		**Anchor count**	
Value	WER	Value	WER
0.96s	6.45	1	6.22
1.92s	6.29	3	6.23
4.00s (default)	6.23	5 (default)	6.23
5.76s	6.26	7	6.20
7.68s	6.23	10	6.21
		15	6.23
		20	6.22

Table 7: Hyperparameter sensitivity at ∼20% GPU retention on Voxtral-Mini-3B, LibriSpeech-Long `test-clean`. Values are truncated WER (↓, ratio 1.2).

Across an 8× range of chunk sizes and a 20× range of anchor-head counts, WER varies by at most about 0.3 points. This indicates that the default configuration is not the result of narrow-range tuning.

D Cross-Dataset Calibration Robustness

To directly test whether head roles overfit the calibration domain, we rerun the full calibration pipeline on 50 LongSpeech French ASR development samples and evaluate the resulting configuration against the original LibriSpeech-English calibration at the same operating point (n_{voice}=90, n_{anchor}=5, target r_{GPU}=0.2). Table 8 reports both calibrations on English LibriSpeech-Long and French LongSpeech ASR.

Calib.	**WER ↓**			**Roles kept**	
	clean	other	FR	anchor	tidal
LS EN	**6.20**	**8.85**	**22.43**	—	—
LS FR	6.24	8.89	23.55	2/5	76/85

Table 8: Cross-dataset calibration robustness on Voxtral-Mini-3B. Both configurations use WhisperX threshold 0.85 and target r_{GPU}=0.2; numbers are truncated WER (↓, ratio 1.2). The LS EN row is an independent re-run of the default configuration, so it differs marginally from its counterpart in Table 5. *Roles kept* reports how many of the anchor and tidal heads chosen by the English calibration are also chosen by the French one.

Switching the calibration language and dataset changes English WER by at most 0.04 points and French WER by 1.12 points, while preserving most tidal heads. As a threshold sanity check, lowering the WhisperX confidence threshold from 0.85 to 0.44 retains 45× more calibration tokens, yet the per-head voice-score correlation remains 0.974 and

Method (r_{GPU}=)	test-clean (WER↓)				test-other (WER↓)			
	0.8	0.6	0.4	0.2	0.8	0.6	0.4	0.2
Backbone: Voxtral-Mini-3B								
Full Cache	6.79				8.86			
Ada-KV	137.46	175.85	196.01	199.24	144.01	181.15	192.60	198.55
AudioKV	6.79	6.31	62.51	192.58	8.85	8.98	66.42	195.53
ArkVale	6.81	6.77	7.12	11.74	8.83	8.85	9.79	14.75
WnW (ours)	**6.72**	**6.71**	**6.64**	**6.23**	**8.84**	**8.84**	**8.81**	**8.87**
Measured audio KV retention								
WnW r_{GPU} (%)	79.79	59.96	40.91	22.25	79.82	59.98	40.93	22.38
WnW $r_{\text{GPU+CPU}}$ (%)	79.79	59.96	40.91	41.52	79.82	59.98	40.93	41.64
Backbone: Qwen2.5-Omni-3B								
Full Cache	13.87				16.80			
Ada-KV	47.45	91.12	109.49	116.04	52.92	98.70	115.53	119.86
AudioKV	14.24	14.54	28.31	131.30	17.62	17.58	34.16	128.27
ArkVale	14.06	14.13	14.12	16.99	16.88	16.95	17.11	21.16
WnW (ours)	**14.72**	**14.90**	**14.88**	**15.31**	**17.80**	**17.77**	**17.84**	**18.42**
Measured audio KV retention								
WnW r_{GPU} (%)	80.26	60.12	40.23	21.78	80.31	60.17	40.28	22.00
WnW $r_{\text{GPU+CPU}}$ (%)	80.26	60.12	40.23	41.81	80.31	60.17	40.28	42.03

Table 5: Full LibriSpeech-Long results across two backbones. Numbers are truncated WER (↓, ratio 1.2). The bottom rows of each backbone block report WnW's measured on-device retention r_{GPU} and combined retention $r_{\text{GPU+CPU}}$ (on-device plus recallable CPU complement).

WER changes by at most 0.31 points relative to the French 0.85 calibration. This suggests that the head roles reflect stable model behavior rather than a narrow calibration artifact.

Calibration is a per-model cost rather than a per-deployment one: head roles must be recomputed for a new backbone, as we do for Voxtral-Small-24B in Appendix B, but no shift in language, task, or domain we study calls for recalibration. Recalibrating on French does not improve French WER. Only 2 of 5 anchors coincide, but Appendix C shows that WER is nearly flat from one to twenty anchors, so the partition tolerates this reshuffling within the audio-grounded pool.

E Calibration Score Details

Voice Score. For each answer token t, WhisperX forced alignment provides a word-level interval $[a_t, b_t]$, converted to audio-token indices $[a_t^{\text{tok}}, b_t^{\text{tok}}) = [\lfloor a_t \cdot r_{\text{tok}} \rfloor, \lfloor b_t \cdot r_{\text{tok}} \rfloor)$. The hit ratio for Q-head (l, h) on token t is

$$\text{hit}_{l,h,t} = \frac{1}{K} \sum_{k=1}^{K} \mathbf{1}\left[\text{topK}_{l,h,t}[k] \in [a_t^{\text{tok}}, b_t^{\text{tok}})\right], \tag{3}$$

where $\text{topK}_{l,h,t}$ is taken over audio positions and K is set to cover roughly one spoken word (K=4 for Voxtral-Mini-3B, K=8 for Qwen2.5-Omni-3B). Averaging over answer tokens (alignment confidence ≥ 0.85) and $|D|$=50 calibration samples yields $\overline{\text{hit}}_{l,h}$, aggregated over GQA groups: $\text{VS}_{l,k} = \frac{1}{g} \sum_{j=1}^{g} \overline{\text{hit}}_{l,(k-1)g+j}$.

Head Sensitivity.

$$S_{l,k,\tau}^{\text{key}} = \left\| \frac{\partial \mathcal{L}}{\partial K_{l,k,\tau}} \right\|, \tag{4}$$

where $\mathcal{L}$ is the answer-token cross-entropy and τ indexes audio positions. The head sensitivity is

$$\text{HS}_{l,k} = \mathbb{E}_D \left[\mathbb{E}_t \left[\text{mean}_{\tau \in \text{audio}} S_{l,k,\tau}^{\text{key}} \right] \right]. \tag{5}$$

F Baseline Hyperparameters

We use the authors' recommended hyperparameters for every baseline unless noted below.

Ada-KV (Feng et al., 2025). `floor_alpha`=0.8, kernel size 7, pooling `max`.

AudioKV (Wang et al., 2026). α=0.5, ρ=0.7, observation window 32, baseline ratio 0.5. We use `min_word_score`=0.85 (rather than the original 0.95) for both AudioKV and WnW so they share the same voice-score calibration; this is the only deviation from author defaults and applies to both.

ArkVale (Chen et al., 2024). Page size 32, top-k recall budget proportional to the target audio KV retention r_{GPU}, CPU mirror of all evicted pages.

AffPool (Xiang et al., 2026). Deep pooling with window size 3 is applied to the first 27 layers of Voxtral-Mini-3B, and input pooling with window size 1 to the last 3 layers. The retention rate is controlled by calibrating the affinity threshold θ on a held-out subset, with measured retention within two percentage points of the target.

G AI Usage

AI assistants were used only for linguistic refinement of the manuscript (clarity of arguments, logical coherence, and grammatical correctness) and for debugging our implementation code. The research idea, experimental design, and analysis of results are entirely the authors' own, who assume full responsibility for the scientific content, conclusions, and integrity of this paper, and confirm compliance with academic ethics and the absence of plagiarism.